

Summary Report

1. Project goals

The goal was to build a C++ project that is stronger than a minimal chess game: a legal chess board plus a neural-network activation visualizer. The project had to satisfy the course requirements for classes, file I/O, multi-file C++ structure, design documentation, user documentation, tests, and final demo material.

2. What we built

We built a native C++17/raylib application with:

- Legal human-vs-human chess play with move highlighting, promotion, undo/redo, status reporting, FEN load/save, and setup editor validation.
- A modular chess core using bitboards, mailbox board access, FEN/SAN support, perft validation, and repetition/draw helpers.
- Three neural-network visualization paths: NNUE, LC0-style CNN, and BT4-style token transformer.
- File I/O for configuration, FEN, PGN support, binary model files, and test reference data.
- A test suite covering chess rules, game history, editor validation, tensor operations, and neural-network paths.

3. Biggest challenges

The hardest part was making dense neural-network data visually useful. A chess board has only 64 squares, but the networks contain thousands or hundreds of thousands of intermediate values. The solution was to show summaries that are still connected to the real computation: board overlays, active feature lists, accumulator bars, convolution heatmaps, token grids, attention summaries, and policy/value outputs.

Another challenge was translating chess-domain ideas from prior Java and web projects into a clean C++ codebase. Translation was not just syntax conversion: ownership, compilation units, header dependencies, memory layout, UI event handling, and CMake integration had to be redesigned.

Raylib visualization also required manual work. Generic layout suggestions from AI tools were often too rough for an interactive desktop program, so the board, panels, controls, validation messages, tutorial demo, and activation views were adjusted by hand.

4. What worked well

The separation between `cnv_core` and `cnv_viz` worked well. The chess and NN logic can be tested without opening a raylib window, while the UI can focus on rendering and user interaction. The linked-list move history also made undo, redo, and move-list display easier to reason about.

The project benefited from existing chess programming experience. Lennart had about five years of chess programming background, including engine work, neural-network training/evaluation experiments, and chess data mining. That made it possible to choose meaningful test cases and avoid common chess-rule mistakes.

5. What we'd do differently

With more time, we would add a true exact LC0 BT4 protobuf weight loader, a PGN importer, a file picker for FEN/PGN paths, and a deeper in-app explanation mode for each architecture. We would also record more automated UI tests if the course environment allowed graphical test automation.

6. Acknowledgements

The implementation was informed by prior personal chess projects:

- <https://github.com/LenniAConrad/chess-rtk>
- <https://github.com/LenniAConrad/chess-web>

Those projects were used as background knowledge and reference material. The submitted runtime code is C++ and does not depend on Java or TypeScript repositories at execution time.